REACT NATIVE

React Native 7 — Euro Converter

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа

Учебный комплект • 2026

Что требуется

- Получить актуальные курсы APILayer Exchange Rates Data API.
- Передать ключ в заголовке `apikey`.
- Заполнить Picker валютами ответа.
- Перевести введённую сумму выбранной валюты в EUR.

Что получится

Expo SDK 54-приложение TypeScript. Ответ сети принимается как `unknown`, карта `Rates` создаётся только после проверки, `Picker<string>` содержит отсортированные коды, а формула EUR-base делит сумму на курс. Секрет хранится лишь в локальном `.env`.

Подготовка на macOS

Нужны Node.js 20.19.4+, npm и Expo Go/Simulator. Проверка: `node --version`, `npm --version`. Получите учебный ключ APILayer; публичная Expo-переменная не является production-секретом, для него нужен backend-proxy.

Создание с нуля

КОД · BASH

```
npx create-expo-app@latest 07-euro-converter --template blank-typescript@sdk-54 --no-agents-md
cd 07-euro-converter
npx expo install @react-native-picker/picker react-dom react-native-web
npm install --save-dev jest@29.7.0 jest-expo@~54.0.18 @testing-library/react-native@13.3.3 react-
test-renderer@19.1.0 @types/jest@29.5.14 @types/node@22.15.3 @types/react@~19.1.10
typescript@~5.9.2
npm ci
cp .env.example .env
```

В `.env` задайте `EXPO_PUBLIC_EXCHANGE_RATES_API_KEY=replace_with_your_key`; никогда не добавляйте этот файл в Git.

Структура файлов

КОД · TEXT

```
07-euro-converter/
  .env.example  env.d.ts  tsconfig.json  package.json
  App.tsx  index.ts  jest.setup.ts
  src/exchange-rates.ts
  __tests__/exchange-rates.test.ts  __tests__/App.test.tsx
```

`package-lock.json`, `dist` и `node_modules` генерируются и не печатаются.

Пошаговая реализация

Ниже приведены дословные листинги всех поддерживаемых вручную файлов.

package.json

КОД · JSON

```json
{
  "name": "fullstack-homework-euro-converter",
  "version": "1.0.0",
  "main": "index.ts",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "test": "jest --watchAll=false",
    "typecheck": "tsc --noEmit",
    "export:web": "expo export --platform web"
  },
  "dependencies": {
    "expo": "~54.0.36",
    "expo-status-bar": "~3.0.9",
    "react": "19.1.0",
    "react-native": "0.81.5",
    "@react-native-picker/picker": "2.11.1",
    "react-dom": "19.1.0",
    "react-native-web": "^0.21.0"
  },
  "devDependencies": {
    "@testing-library/react-native": "13.3.3",
    "jest": "29.7.0",
    "jest-expo": "~54.0.18",
    "react-test-renderer": "19.1.0",
    "@types/jest": "29.5.14",
    "@types/node": "22.15.3",
    "@types/react": "~19.1.10",
    "typescript": "~5.9.2"
  },
  "jest": {
    "preset": "jest-expo",
    "setupFilesAfterEnv": ["<rootDir>/jest.setup.ts"]
  },
  "private": true
}
```

tsconfig.json, env.d.ts, .env.example

КОД · JSON

```json
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true
  },
  "include": ["**/*.ts", "**/*.tsx", "env.d.ts"]
}
```

КОД · TS

```ts
declare namespace NodeJS {
  interface ProcessEnv {
    readonly EXPO_PUBLIC_EXCHANGE_RATES_API_KEY?: string;
  }
}
```

КОД · DOTENV

```
EXPO_PUBLIC_EXCHANGE_RATES_API_KEY=
```

`strict` проверяет все файлы. Переменная необязательна, поэтому отсутствие ключа обрабатывается до сети.

src/exchange-rates.ts

КОД · TS

```ts
const endpoint = 'https://api.apilayer.com/exchangerates_data/latest?base=EUR';

export type Rates = Readonly<Record<string, number>>;

export type RatesResult =
  | { readonly ok: true; readonly rates: Rates }
  | { readonly ok: false; readonly error: string };

export type ConversionResult =
  | { readonly ok: true; readonly value: number }
  | { readonly ok: false; readonly error: string };

export type FetchResponse = Readonly<{
  ok: boolean;
  json: () => Promise<unknown>;
}>;

export type FetchImplementation = (
  input: string,
  init?: RequestInit,
) => Promise<FetchResponse>;

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}

function parseRates(payload: unknown): Rates | null {
  if (!isRecord(payload) || !isRecord(payload.rates)) {
    return null;
  }

  const entries = Object.entries(payload.rates);
  if (entries.length === 0) {
    return null;
  }

  const rates: Record<string, number> = {};
  for (const [code, rate] of entries) {
    if (code.trim() === '' || typeof rate !== 'number' || !Number.isFinite(rate) || rate <= 0) {
      return null;
    }
    rates[code] = rate;
  }

  return rates;
}

export function currencyCodes(rates: Rates): string[] {
  return Object.keys(rates).sort();
}

export async function fetchEuroRates(
  apiKey: string | undefined,
  fetchImpl: FetchImplementation = fetch,
): Promise<RatesResult> {
```

```
  const normalizedApiKey = apiKey?.trim();
  if (!normalizedApiKey) {
    return { ok: false, error: 'Add EXPO_PUBLIC_EXCHANGE_RATES_API_KEY to .env.' };
  }

  try {
    const response = await fetchImpl(endpoint, {
      headers: { apikey: normalizedApiKey },
    });
    if (!response.ok) {
      return { ok: false, error: 'Could not load rates. Check the API key and connection.' };
    }

    const payload: unknown = await response.json();
    const rates = parseRates(payload);
    return rates
      ? { ok: true, rates }
      : { ok: false, error: 'The rates response is invalid.' };
  } catch {
    return { ok: false, error: 'Could not load rates. Check the API key and connection.' };
  }
}

export function convertToEuro(
  amountText: string,
  currency: string,
  rates: Rates,
): ConversionResult {
  const normalized = amountText.trim().replace(',', '.');
  const amount = Number(normalized);
  if (normalized === '' || !Number.isFinite(amount)) {
    return { ok: false, error: 'Enter a valid amount.' };
  }

  const rate = rates[currency];
  if (!Number.isFinite(rate) || rate <= 0) {
    return { ok: false, error: 'The selected currency has no rate.' };
  }

  return { ok: true, value: amount / rate };
}
```

`unknown → isRecord → parseRates → Rates` — граница доверия API. `FetchImplementation` позволяет подать в тест настоящий типизированный fake. Discriminated unions отделяют error path от успеха. При `USD: 1.25` 125.50 USD делятся на 1.25 и дают 100.40 EUR.

App.tsx

КОД · TSX

```
import { useEffect, useState } from 'react';
import { Button, Text, TextInput, View } from 'react-native';
import { Picker } from '@react-native-picker/picker';
import {
  convertToEuro,
  currencyCodes,
  fetchEuroRates,
  type FetchImplementation,
  type Rates,
} from './src/exchange-rates';

type AppProps = Readonly<{
  fetchImpl?: FetchImplementation;
}>;
```

```
export default function App({ fetchImpl }: AppProps): React.JSX.Element {
  const [amount, setAmount] = useState<string>('');
  const [rates, setRates] = useState<Rates | null>(null);
  const [codes, setCodes] = useState<string[]>([]);
  const [currency, setCurrency] = useState<string>('');
  const [loading, setLoading] = useState<boolean>(true);
  const [errorMessage, setErrorMessage] = useState<string>('');
  const [result, setResult] = useState<string>('');

  useEffect(() => {
    async function loadRates(): Promise<void> {
      const rawApiKey: unknown = process.env.EXPO_PUBLIC_EXCHANGE_RATES_API_KEY;
      const apiKey = typeof rawApiKey === 'string' ? rawApiKey.trim() : undefined;
      const response = await fetchEuroRates(apiKey, fetchImpl);

      setLoading(false);
      if (!response.ok) {
        setErrorMessage(response.error);
        return;
      }

      const nextCodes = currencyCodes(response.rates);
      setRates(response.rates);
      setCodes(nextCodes);
      setCurrency(nextCodes[0] ?? '');
    }

    void loadRates();
  }, [fetchImpl]);

  function convert(): void {
    const conversion = convertToEuro(amount, currency, rates ?? {});
    if (!conversion.ok) {
      setErrorMessage(conversion.error);
      return;
    }

    setErrorMessage('');
    setResult(`${conversion.value.toFixed(2)} EUR`);
  }

  return (
    <View style={{ gap: 12, padding: 16 }}>
      <Text>Amount</Text>
      <TextInput
        accessibilityLabel="Amount"
        keyboardType="decimal-pad"
        placeholder="e.g. 125.50"
        value={amount}
        onChangeText={setAmount}
      />
      {loading ? <Text accessibilityLiveRegion="polite">Loading rates…</Text> : null}
      {codes.length > 0 ? (
        <Picker<string>
          accessibilityLabel="Currency picker"
          selectedValue={currency}
          onValueChange={(value: string): void => setCurrency(value)}
        >
          {codes.map((code) => <Picker.Item key={code} label={code} value={code} />)}
        </Picker>
      ) : null}
      <Button title="CONVERT TO EUR" disabled={loading || rates === null} onPress={convert} />
      {errorMessage ? (
        <Text
          accessibilityLabel="Converter error"
          accessibilityRole="alert"
          accessibilityLiveRegion="assertive"
```

```
        >
          {errorMessage}
        </Text>
      ) : null}
      {result ? (
        <Text accessibilityLabel="Conversion result" accessibilityLiveRegion="polite">
          {result}
        </Text>
      ) : null}
    </View>
  );
}
```

`useEffect` загружает курсы после первого рендера. Типы state запрещают использовать карту до загрузки; `Picker<string>` связывает значение и callback. Ошибка загрузки или конверсии не делает сетевой запрос повторно и объявляется как alert.

index.ts и jest.setup.ts

КОД · TS

```
import { registerRootComponent } from 'expo';
import App from './App';

registerRootComponent(App);
```

КОД · TS

```
import type { ComponentProps, ComponentType, ReactElement, ReactNode } from 'react';
import { Picker as NativePicker } from '@react-native-picker/picker';

type mockPickerProps = ComponentProps<typeof NativePicker<string>>;
type mockPickerItemProps = ComponentProps<typeof NativePicker.Item>;
type mockPickerTestProps = mockPickerProps & { readonly children?: ReactNode };
type mockPickerItemTestProps = mockPickerItemProps;
type mockPickerComponent = ComponentType<mockPickerTestProps> & {
  Item: ComponentType<mockPickerItemTestProps>;
};

jest.mock('@react-native-picker/picker', () => {
  const React = jest.requireActual<typeof import('react')>('react');
  const { View } = jest.requireActual<typeof import('react-native')>('react-native');
  type mockTestViewProps = ComponentProps<typeof View> & mockPickerTestProps &
mockPickerItemTestProps;
  const mockTestView = View as ComponentType<mockTestViewProps>;

  const mockPicker = (({ children, ...props }: mockPickerTestProps): ReactElement => (
    React.createElement(mockTestView, props, children)
  )) as mockPickerComponent;
  mockPicker.Item = (props: mockPickerItemTestProps): ReactElement =>
React.createElement(mockTestView, props);

  return { Picker: mockPicker };
});
```

Entry point регистрирует Expo-компонент. Jest mock заменяет только нативный Picker и сохраняет его props для доступных UI-тестов.

__tests__/exchange-rates.test.ts

КОД · TS

```
import {
```

```
  convertToEuro,
  currencyCodes,
  fetchEuroRates,
  type FetchImplementation,
  type FetchResponse,
} from '../src/exchange-rates';

function response(payload: unknown, ok = true): FetchResponse {
  return { ok, json: async (): Promise<unknown> => payload };
}

test('loads validated numeric rates with the APILayer apikey header', async () => {
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  fetchImpl.mockResolvedValue(response({ rates: { USD: 1.25, EUR: 1 } }));

  await expect(fetchEuroRates('test-key', fetchImpl)).resolves.toEqual({
    ok: true,
    rates: { USD: 1.25, EUR: 1 },
  });
  expect(fetchImpl).toHaveBeenCalledWith(
    'https://api.apilayer.com/exchangerates_data/latest?base=EUR',
    { headers: { apikey: 'test-key' } },
  );
});

test.each<unknown>([
  null,
  {},
  { rates: null },
  { rates: [] },
  { rates: {} },
  { rates: { USD: '1.25' } },
  { rates: { USD: -1 } },
])('rejects an invalid rates payload: %p', async (payload) => {
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  fetchImpl.mockResolvedValue(response(payload));

  await expect(fetchEuroRates('test-key', fetchImpl)).resolves.toEqual({
    ok: false,
    error: 'The rates response is invalid.',
  });
});

test('reports absent credentials before making a request', async () => {
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();

  await expect(fetchEuroRates(undefined, fetchImpl)).resolves.toEqual({
    ok: false,
    error: 'Add EXPO_PUBLIC_EXCHANGE_RATES_API_KEY to .env.',
  });
  expect(fetchImpl).not.toHaveBeenCalled();
});

test('reports an HTTP or network error', async () => {
  const httpFetch = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  httpFetch.mockResolvedValue(response({}, false));
  const offlineFetch = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  offlineFetch.mockRejectedValue(new Error('offline'));

  await expect(fetchEuroRates('test-key', httpFetch)).resolves.toEqual({
    ok: false,
    error: 'Could not load rates. Check the API key and connection.',
  });
  await expect(fetchEuroRates('test-key', offlineFetch)).resolves.toEqual({
    ok: false,
    error: 'Could not load rates. Check the API key and connection.',
  });
```

```
});

test('sorts currency codes and converts comma decimals using the EUR-base rate', () => {
  const rates = { USD: 1.25, EUR: 1 };

  expect(currencyCodes(rates)).toEqual(['EUR', 'USD']);
  expect(convertToEuro('125,50', 'USD', rates)).toEqual({ ok: true, value: 100.4 });
});

test.each([
  ['', 'USD', { USD: 1.25 }],
  ['x', 'USD', { USD: 1.25 }],
  ['20', 'GBP', { USD: 1.25 }],
])('rejects an invalid conversion input', (amount, currency, rates) => {
  expect(convertToEuro(amount, currency, rates)).toEqual(
    amount === '' || amount === 'x'
      ? { ok: false, error: 'Enter a valid amount.' }
      : { ok: false, error: 'The selected currency has no rate.' },
  );
});
```

__tests__/App.test.tsx

КОД · TSX

```
import React from 'react';
import { fireEvent, render, waitFor } from '@testing-library/react-native';
import App from '../App';
import type { FetchImplementation, FetchResponse } from '../src/exchange-rates';

function response(payload: unknown, ok = true): FetchResponse {
  return { ok, json: async (): Promise<unknown> => payload };
}

const apiKeyName: string = 'EXPO_PUBLIC_EXCHANGE_RATES_API_KEY';

beforeEach(() => {
  process.env[apiKeyName] = 'test-key';
});

afterEach(() => {
  delete process.env[apiKeyName];
});

test('loads picker codes and converts USD to EUR', async () => {
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  fetchImpl.mockResolvedValue(response({ rates: { USD: 1.25, EUR: 1 } }));
  const ui = render(<App fetchImpl={fetchImpl} />);

  expect(ui.getByText('Loading rates…')).toHaveProp('accessibilityLiveRegion', 'polite');
  await waitFor(() => expect(ui.getByLabelText('Currency picker')).toBeTruthy());
  fireEvent(ui.getByLabelText('Currency picker'), 'valueChange', 'USD');
  fireEvent.changeText(ui.getByLabelText('Amount'), '125,5');
  fireEvent.press(ui.getByRole('button', { name: 'CONVERT TO EUR' }));

  expect(ui.getByLabelText('Conversion result')).toHaveTextContent('100.40 EUR');
});

test('shows an accessible missing-key error without requesting rates', async () => {
  delete process.env[apiKeyName];
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  const ui = render(<App fetchImpl={fetchImpl} />);

  await waitFor(() => expect(ui.getByRole('alert', { name: 'Converter error'
})).toHaveTextContent(
    'Add EXPO_PUBLIC_EXCHANGE_RATES_API_KEY to .env.',
```

```
  ));
  expect(fetchImpl).not.toHaveBeenCalled();
});

test('shows an accessible error when the API returns no usable rates', async () => {
  const fetchImpl = jest.fn<Promise<FetchResponse>, Parameters<FetchImplementation>>();
  fetchImpl.mockResolvedValue(response({ rates: {} }));
  const ui = render(<App fetchImpl={fetchImpl} />);

  await waitFor(() => expect(ui.getByRole('alert', { name: 'Converter error'
})).toHaveTextContent(
    'The rates response is invalid.',
  ));
  expect(ui.queryByLabelText('Currency picker')).not.toBeOnTheScreen();
  expect(ui.getByRole('button', { name: 'CONVERT TO EUR' })).toBeDisabled();
});
```

Tests exercise success, external malformed data, missing key, HTTP/network errors, screen state and accessibility.

Запуск и тесты

КОД · BASH

```
npm ci
npm run typecheck
npm test -- --runInBand
npx expo-doctor
npm run export:web
npm start -- --clear
```

Ручная проверка

Введите USD и `125,5`; при курсе 1.25 увидите 100.40 EUR. Проверьте пустую строку, `abc`, отключённую сеть и отсутствующий `.env`; запрос без ключа не отправляется.

Типичные ошибки

Не приводите JSON к `Rates` через `as`; не используйте `any`; не умножайте вместо деления; не помещайте ключ в URL, исходник или `.env.example`.

Вопросы и ответы

Почему `unknown`? Удалённый сервер не типизирован. **Почему `Picker<string>`?** Он сохраняет только строковый код валюты. **Почему public key не секрет?** Его получает клиентское приложение.

Защита

Покажите цепочку `unknown JSON → parseRates → Rates → Picker<string> → convertToEuro`, объясните формулу EUR-base, запустите typecheck и офлайн-тесты, затем web export.

Чек-лист сдачи

- ☐ `.env` игнорируется, пример пустой.
- ☐ Ключ передаётся заголовком `apikey`.
- ☐ Payload, сумма, курс и ошибки валидируются.
- ☐ Исходники и tests — TS/TSX.
- ☐ Проходят typecheck, Jest, Expo Doctor и web export.